

EE5108 Digital Twins: Deep Q-Learning for Lunar Lander Stabilization

Balu Monish

March 15, 2026

Abstract

This report details the implementation and hyperparameter optimization of a Deep Q-Network (DQN) agent trained to solve the Gymnasium LunarLander-v3 environment. The simulation acts as a digital twin for spacecraft stabilization. Three distinct hyperparameter configurations—Baseline, Fast Learner, and Precision Pilot—were evaluated to understand the impact of learning rate, epsilon decay, and target network update frequency on policy convergence.

1 Introduction

Reinforcement Learning (RL) provides a robust framework for developing control policies in highly stochastic environments. In this project, a DQN agent was trained to map an 8-dimensional state vector (representing coordinates, velocities, angle, and leg contact) to a 4-dimensional discrete action space (main engine, left engine, right engine, do nothing). The objective was to achieve an average score of ≥ 250.0 over 100 consecutive episodes, requiring the agent to balance fuel efficiency with safe, centered landings.

2 Methodology

The agent utilizes a multi-layer perceptron with two hidden layers of 64 nodes each, activated by ReLU functions. To stabilize training, the architecture employs Experience Replay (buffer size of 100,000) and a dual-network setup (Local and Target networks). The baseline hyperparameter limits were set to 2000 maximum episodes and 1000 maximum timesteps per episode.

3 Results

Three configurations were tested to observe the convergence dynamics.

3.1 Configuration 1: Baseline

The Baseline model utilized a learning rate of $5e-4$, an epsilon decay of 0.997, and updated the target network every 4 steps. This configuration reached the 250.0 threshold, officially solving the environment in 1697 episodes with a final average score of 250.40. The learning curve showed steady, predictable improvement.

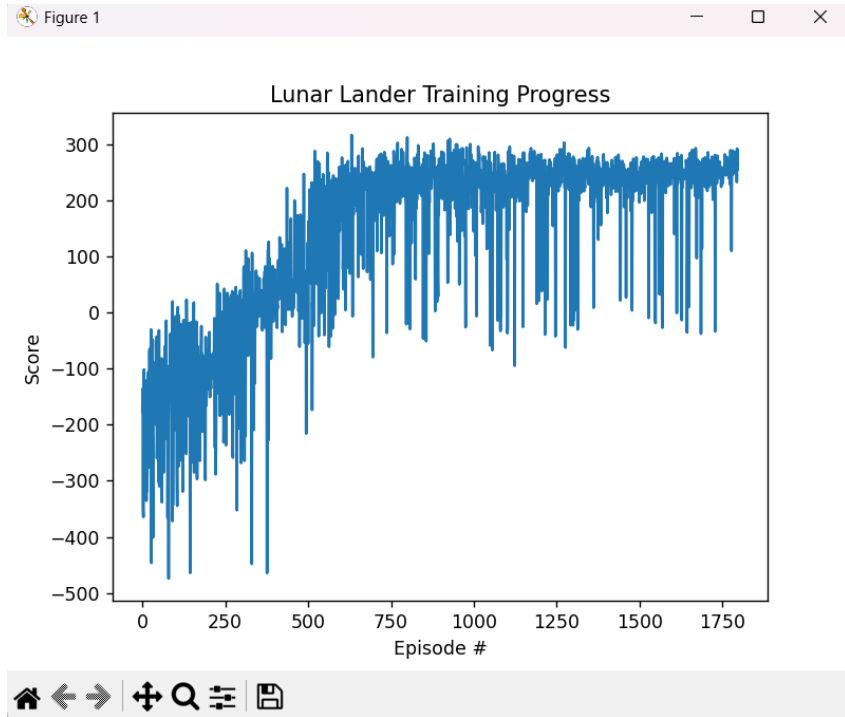


Figure 1: Baseline Training Progress

3.2 Configuration 2: The Fast Learner

Designed for rapid convergence, this model used an aggressive learning rate of $1e - 3$ and a faster epsilon decay of 0.990. The environment was solved in just 1389 episodes with an average score of 250.1. However, the early stages of training were highly volatile.

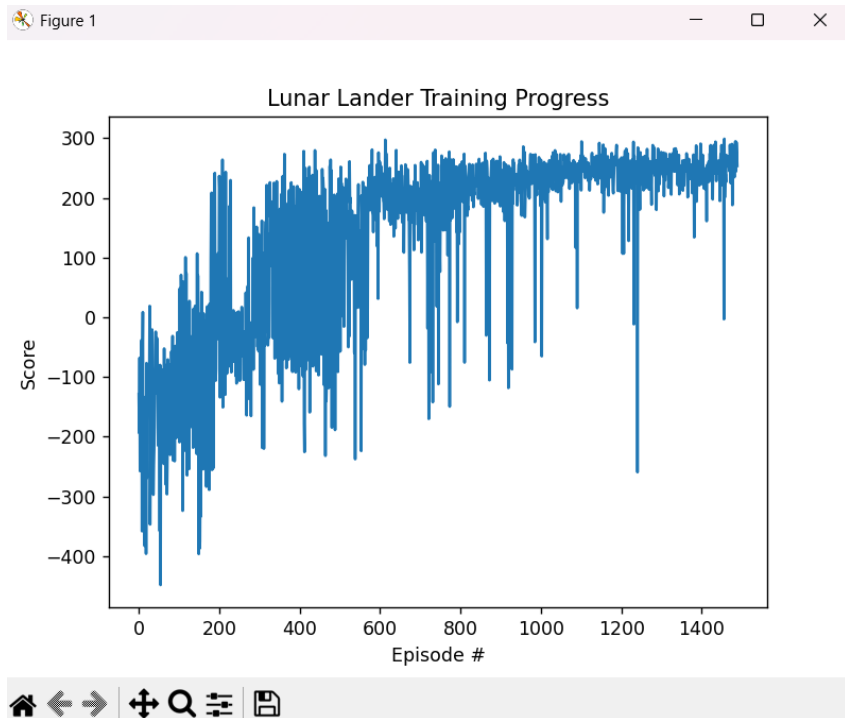


Figure 2: Fast Learner Training Progress

3.3 Configuration 3: Precision Pilot

Intended to maximize stability, this model reduced the learning rate to $1e - 4$ and extended epsilon decay to 0.999. This model failed to solve the environment within the 2000-episode limit (DNF).

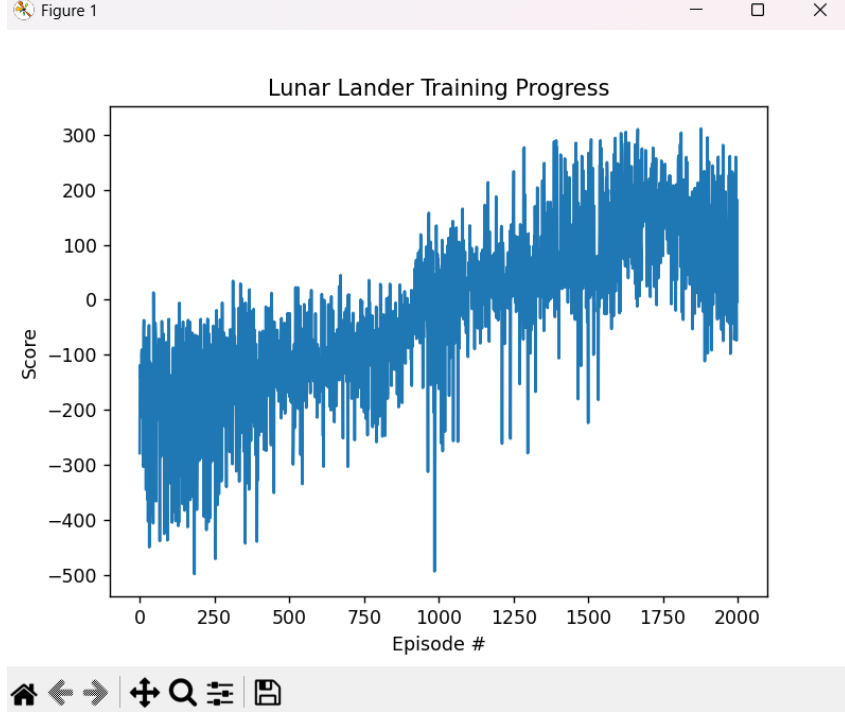


Figure 3: Precision Pilot Training Progress

4 Analysis and Discussion

4.1 Training Trends and Milestones

Tracking the average scores at specific episode intervals highlights the distinct learning trajectories generated by each hyperparameter configuration. Table 1 summarizes these milestones.

Episode	Baseline (Score)	Fast Learner (Score)	Precision Pilot (Score)
100	-174.20	-155.70	-182.00
500	58.43	86.81	-127.70
1000	230.97	213.42	-17.62
1500	232.40	250.11 (Ep 1489)	82.74
Final	250.40 (Ep 1797)	250.10 (Ep 1389)	92.98 (Ep 2000)

Table 1: Average Score Progression per Experiment

The Baseline experiment showed a steady and consistent climb, breaking into positive scores around episode 500 and stabilizing above 200 by episode 700. The Fast Learner experiment demonstrated a much steeper initial learning curve, achieving positive scores by episode 400 and maintaining a high trajectory. However, it exhibited higher variance in the later stages. The Precision Pilot struggled significantly, remaining in the negative score range until episode 1200. Although it showed a late surge, peaking at 158.12 at episode 1700, it suffered a catastrophic collapse in performance towards the end of the training limit.

4.2 Exploration Dynamics and Learning Speed

The trade-off between learning speed and final performance is heavily governed by the epsilon decay rate. An aggressive decay (0.990 in the Fast Learner) forces the agent to exploit its neural network earlier. While this drastically reduced the total episodes required to solve the environment, it introduced significant instability. In contrast, the Precision Pilot’s slow decay (0.999) prioritized deep exploration. However, this caused the agent to spend too much time taking random actions, ultimately preventing it from solidifying a winning policy within the 2000-episode limit. The Baseline decay (0.997) provided the optimal balance.

4.3 Target Network Stability

The frequency of target network updates plays a critical role in preventing divergence. If the target network is updated too frequently, the agent attempts to map its Q-values to a constantly moving target, leading to the highly jagged, oscillating scores observed in the Fast Learner’s curve. Conversely, updating too rarely can stall the learning process entirely, as the local network minimizes loss against an outdated baseline.

4.4 Dominant Hyperparameters

Upon reviewing the three profiles, the learning rate combined with the epsilon decay schedule had the most profound impact on overall performance. The stark contrast between the Fast Learner’s rapid completion and the Precision Pilot’s failure to converge demonstrates that while a smaller learning rate theoretically provides a safer descent down the loss gradient, it requires a proportionally larger episode allowance to compensate for the reduced step size.

4.5 Observed Failure Modes

Several failure modes were observed during inference and training. The most notable was “catastrophic forgetting,” evidenced in the Precision Pilot’s curve, where performance peaked before inexplicably collapsing by episode 2000. Additionally, agents frequently demonstrated a local optimum trap: hovering excessively or landing outside the target flags to guarantee the “safe landing” reward while avoiding the risk of crashing associated with centering the craft.

4.6 Learned Policy vs. Intuition

Human intuition suggests that an optimal landing involves a smooth, direct vertical descent. However, the RL policy often diverges from this path. Because the agent is penalized heavily for fuel consumption but rewarded heavily for any safe leg contact, the trained agent frequently adopts a “swooping” trajectory, essentially free-falling and executing a hard deceleration at the last possible moment. While visually alarming compared to human piloting, it is a mathematically superior exploitation of the reward structure.

5 Conclusion

Developing a digital twin using Deep Q-Learning highlights the sensitivity of neural networks to hyperparameter configurations. The Baseline model proved superior by effectively balancing exploration with stable network updates, successfully managing the stochastic physics of the environment.